

Project Handoff: "Grandma" — Grumpy Usage Tracker & Workflow Visualizer

Owner: Daniel **Builder:** Claude Code (multi-agent, Decision Brief approval flow) **Status:** Phase 0 — spec approved, ready to scaffold

1. Concept

Grandma is a desktop companion with a grumpy-but-loving grandmother personality. She does two jobs:

1. **Usage tracking** — watches Claude Code token/cost burn across sessions and agents, and complains about it.
2. **Workflow visualization** — shows the live multi-agent pipeline (which agent is active, pending handoffs, Decision Brief queue) so Daniel can see orchestration state at a glance.

She is built in two phases. Phase 1 lives in the terminal (statusline + hooks = the data layer). Phase 2 promotes her to a persistent desktop app that consumes the same data layer. **Phase 1's event log IS Phase 2's API.** Do not let the phases diverge.

2. Personality Contract (applies to all phases)

- Tone: dry, disapproving, secretly proud. Never mean about the *user*, only about token waste, idle agents, and sloppy handoffs.
- One-liners only in the statusline ($\leq$ terminal width; `COLUMNS` env var is provided to statusline scripts).
- Longer grumbling allowed in the desktop app ("Three agents running and not one of them has finished anything. In my day we had ONE context window and we were grateful.").
- Personality strings live in a single `grandma_lines.json` keyed by trigger (e.g. `burn_high`, `idle_agent`, `handoff_clean`, `brief_pending`, `session_start`, `budget_ok`). No hardcoded quips in logic code.
- Severity tiers: `content` (usage < 50%), `suspicious` (50–80%), `disappointed` (> 80%), `slipper_thrown` (rate limit imminent).

3. Verified Platform Facts (do not re-derive; confirmed against docs June 2026)

- **Statusline:** configured in `~/.claude/settings.json` as `"statusLine": {"type": "command", "command": "<script>", "padding": 0}`. Claude Code pipes JSON to the script via stdin including `session_id`, `transcript_path`, `cwd`, `model.{id,display_name}`, `workspace`, `cost.total_cost_usd`, and context-window usage fields. First line of stdout becomes the statusline; ANSI colors supported; updates at most every ~300ms. Requires workspace trust acceptance. `COLUMNS/LINES` env vars available for width-aware rendering.
 - **Hooks:** configured under `"hooks"` in settings; events include `SessionStart`, `PreToolUse`, `PostToolUse`, `Stop`, `SubagentStop`, with matcher patterns (e.g. `"matcher": "Task"` for subagent launches). Hooks receive JSON on stdin and can run shell commands. Async/non-blocking hooks are supported — use them; Grandma must never slow the pipeline.
 - **Session transcripts:** JSONL files under `~/.claude/projects/<project-hash>/.transcripts/`. The statusline JSON's `transcript_path` points at the live one.
 - Verify current hook event names/fields at build time against <https://code.claude.com/docs> (they version frequently).
-

4. Phase 1 — Terminal Data Layer (statusline + hooks)

4.1 Deliverables

1. `~/.claude/grandma/` directory:
 - `grandma_statusline.sh` (or `.mjs`) — reads stdin JSON, reads state file, prints one grumpy line with usage bar.
 - `grandma_hook.mjs` — single hook entrypoint, dispatched by event name (keep `settings.json` hook config thin).
 - `state/events.ndjson` — append-only event log. **This is the contract with Phase 2.**
 - `state/current.json` — latest snapshot (active session, active agents, burn rate, brief queue) for cheap reads.
 - `grandma_lines.json` — personality strings.
2. Settings patch (provide as a snippet for Daniel to merge, do not overwrite his `settings.json`):
 - `statusLine` → `grandma_statusline.sh`

- hooks: `SessionStart`, `Stop`, `SubagentStop`, `PostToolUse` (matcher `Task`)
→ `grandma_hook.mjs` (async where supported).

4.2 Event schema (`events.ndjson`, one JSON object per line)

```
{
  "ts": "2026-06-12T14:03:22Z",
  "session_id": "abc123",
  "project": "dugout-dynasty",
  "event": "subagent_stop | session_start | tool_use | handoff | brief_created | brief_approved",
  "agent": "scout-agent",
  "tokens_in": 12345,
  "tokens_out": 6789,
  "cost_usd": 0.21,
  "meta": {}
}
```

- `handoff` events: derive from `Task` tool `PostToolUse` (parent → subagent) and `SubagentStop` (subagent → parent).
- `brief_*` events: parse Decision Brief files in the Dugout Dynasty repo (Daniel's approval system writes briefs to a known directory — confirm path with him at kickoff; watch it with the hook on `Stop` or a lightweight fs poll).
- Token/cost numbers: prefer statusline stdin fields; fall back to summing the transcript JSONL.

4.3 Statusline output format

🤖 [Opus] ██████████░░░░░░ 52% ctx | \$1.84 today | 2 agents up | "Burning daylight AND tokens, I see."

- Quip selected by severity tier; rotate within tier; never repeat the last-shown quip.
- Truncate gracefully to `COLUMNS`.

4.4 Acceptance criteria

- Statusline renders within 100ms (read `current.json`, never parse full transcripts on the hot path).
 - Hooks never block: total hook wall time < 200ms or async.
 - Killing/removing Grandma files must not break Claude Code (graceful no-op if state dir missing).
 - Works with multiple concurrent sessions (key state by `session_id`).
-

5. Phase 2 — Desktop App ("Grandma lives in the desktop")

5.1 Stack

- **Tauri v2 + React** (small binary, low idle RAM — Grandma is always-on; Electron is a fallback only if Tauri's fs-watch is problematic on Daniel's machine).
- macOS menu-bar item + a floating "Grandma window" (toggleable, always-on-top option).

5.2 Design source (Claude Design import)

- The UI design is authored in **Claude Design** (Anthropic Labs) and exported as a **Claude Code handoff bundle**. That bundle is the single source of truth for visuals: layout, components, colors, typography, Grandma's avatar/expressions.
- Builder responsibilities on import:
 1. Unpack the bundle into the repo (`/design/` directory); do not hand-redraw components that exist in the bundle.
 2. Map bundle components into the Tauri + React shell. Claude Design exports are web/HTML-oriented, so they should drop into the React webview with minimal adaptation — flag any component that doesn't translate cleanly rather than silently restyling it.
 3. **Rewire all data bindings**: replace any mock/placeholder data in the design with live reads from `events.ndjson` / `current.json` (Section 5.3 contract). No mock data may survive to the MVP.
 4. Replace any placeholder copy in the design with lookups into `grandma_lines.json` — personality stays centralized even if the design ships with sample quips.
- If the design and this spec conflict on *visuals*, the design wins. If they conflict on *data flow*, *architecture*, or *personality sourcing*, this spec wins. Raise a Decision Brief if the conflict is non-trivial.

5.3 Data source

- Watch `~/ .claude/grandma/state/events.ndjson` and `current.json` via fs events (tail, don't re-read).
- No direct transcript parsing in the app; the hook layer owns ingestion. If the app needs new data, extend the event schema, don't bypass it.

5.4 Views

1. **Grandma face panel** — avatar whose expression maps to severity tier; current quip; today's cost; burn-rate sparkline.
2. **Workflow graph** — live node graph of orchestrator + subagents built from `handoff` events. Active agent pulses; idle > N min gets Grandma's side-eye annotation. Per-node token totals.
3. **Decision Brief queue** — pending briefs with age; one-click "open in terminal/editor" (deep link via `cwd` + file path). Grandma nags when a brief sits > 30 min.
4. **History** — per-day cost/token chart, per-agent breakdown, top offenders ("Your scout agent ate 40% of the groceries this week").

5.5 Acceptance criteria

- UI matches the imported Claude Design bundle (visual source of truth); deviations require a Decision Brief.
 - < 1s latency from hook event → UI update.
 - Idle CPU ~0%, RAM < 150MB.
 - Survives Claude Code restarts and multiple projects; project switcher in UI.
 - All copy routes through `grandma_lines.json` (shared file, single source of personality).
-

6. Build Order & Decision Briefs

Each numbered item below = one Decision Brief for Daniel's approval before implementation:

1. **DB-1:** Confirm Decision Brief directory path + brief file format; finalize event schema.
2. **DB-2:** Phase 1 hook + statusline implementation plan (language choice, settings patch).
3. **DB-3:** Phase 1 ship + 2-day soak test criteria.
4. **DB-4:** Import & audit the Claude Design bundle — inventory components, confirm they map to Tauri v2 + React, list required data bindings against the event schema, flag gaps (views in spec but missing from design, or vice versa).
5. **DB-5:** Phase 2 MVP (views 1–2 wired to live data), then **DB-6:** views 3–4.

7. Non-goals

- No cloud sync, no telemetry leaving the machine.
- No modifying Claude Code behavior (read-only observer; hooks must always exit 0 / non-blocking).
- No per-message content analysis — Grandma judges metadata, not your prompts. She's nosy but not *that* nosy.